Who's Who and What's What?

Steven Breger



Figure 1: Isn't Gemini amazing?

1 The digital ID problem

How does a police officer know who you are? Well, that's easy. You show them an ID. In fact, it's how you prove your identity to almost anyone, whether that's a bank, a bar, or an airport.

But, when your computer connects to a server, how does it truly know who that server is? ID cards prevent people from getting away with lying about their identity, but how does your computer verify the identity of the server without just taking its word? Moreover, how does the server verify your computers identity without taking its word?

Just like police don't want to arrest the wrong people, banks don't want to give others access to your money, bars don't want underage people drinking, and airports want to verify who you are, servers and clients need to know that the connection they have established is legitimate and is with whatever entity they thought it was. If these measures don't exist, a hacker could just intercept a connection and steal sensitive data.

So, now that we see the benefits of creating a digital version of identity verification, the question becomes how. How can this be achieved? Well, your first thought might be to generate a unique ID card for each person. We could generate a random 2048-bit string and have that be an ID (something like 10111011101000110101111110000111...). Although this does work, there's a major problem. Unlike a physical ID card, which is unique and has anti-counterfeit protections, as soon as you give this binary string to someone else, they could pretend to be you by copy and pasting this string to someone else. That's why for digital identity verification we have to be a bit more clever. Take a moment to make a rough guess for how we could implement digital ID cards before moving on.

2 More Intuition

Note that from here on out, we will switch to using the word "key" instead of "ID" as that is the more common term within computer science, but that keys can essentially be thought of as ID cards. Okay, have you thought about it? Well, the answer is that we need a private key. Something that only you have and that you can't give to anyone else. This, however, gives rise to another problem. How can anyone verify your identity if you can't share your private key with others? To solve this problem, we would generate a corresponding public key that goes along with the private key. Once you share your public key with someone, they would then hopefully somehow be able to verify that you have the corresponding private key. Think of the public key as saying "I claim my identity to be X," after which whoever is verifying your identity can test that you really are X by verifying that you have the corresponding private key.

The careful reader may have caught a few problems with the above proposal. Namely,

1. Whoever is verifying your identity has to know that the public key you provided is actually the key corresponding to you. Otherwise, a hacker could just generate their own public-private key pair, claim that their public key corresponds to you, and steal your identity.

2. It must be impossible to obtain the private key from the public key. Moreover, it must be impossible to obtain the private key while verifying that you have the private key. So, in essence, you have to prove that you have the private key without actually letting anyone get any information about it.

The first of these problems will be tackled later on and the second is solved by the nature of the math that we will employ to implement this algorithm.

3 The Math Under the Hood

Here's the key mathematical idea that we will use to design our algorithm: the discrete root problem is really hard to solve. In other words, given a, r, and n, it is very difficult to find a g (assuming that such a g exists), such that $g^a \equiv r \pmod{n}$.

It's pretty clear how we can begin to encrypt data with this. Given data d, we can encrypt it with public key (a, n) by computing $d' \equiv d^a \pmod{n}$. Someone seeing d' wouldn't easily be able to compute d by taking the ath root. It's computationally infeasible: taking an ath is harder than raising something to the ath power for positive integers a.

Okay, now that the original data is impossible to retrieve from the encrypted data with the public key, let's try to find a private key that can decrypt our data. It seems like the most logical way to do this is to take another exponent. So, we can try to compute a b such that

$$\begin{aligned} & d'^b \equiv d \pmod{n} \\ \iff & d^{ab} \equiv d^1 \pmod{n} \end{aligned}$$

If we can guarantee that $\varphi(n) \mid ab - 1$, then the above equation will be true by Euler's totient theorem. Looking at this condition in light of modular arithmetic, it is sufficient that the private key involves the modular inverse of a modulo $\varphi(n)$. So, our private key becomes (b, n) where $b \equiv a^{-1} \pmod{\varphi(n)}$.

Aside from the issue of everything having to be relatively prime and "nice" for these modular inverses and exponentials to work, we also have another problem. If someone has the public key a and $\varphi(n)$, then they would be able to compute b by using the Euclidean algorithm to compute a modular inverse. So, it follows that we must make $\varphi(n)$ impossible to obtain from n (since n must be in the public key). If $n = p$ for a prime p, obtaining $\varphi(n)$ is easy, just compute $p - 1$. However, if we simply let $n = pq$ for primes p and q, $\varphi(n) = (p-1)(q-1) = pq - p - q + 1 = n - p - q + 1$. Aha! Although this is easy for us to compute if we generate p and q first, in order to find $\varphi(n)$ an attacker must first find p and q just from n. In other words, an attacker would have to factor n. But, for large n this is incredibly slow and computationally infeasible!

Notice that our decryption relied on the fact that $d^{ab} \equiv d^1 \pmod{n}$ when $\varphi(n) \mid ab - 1$. Usually, we only consider these types of statements for d relatively prime to n. However, it turns out that because this

equation has a factor of d on both sides, it is actually true for any value of d when n is square-free). To see this, let $n = p_1 p_2 \cdots p_k$ for primes p_i. By Chinese remainder theorem, our equation is true if and only if

$$d^{ab} \equiv d^1 \pmod{p_i}$$

for each i. Since $p_i - 1 \mid \varphi(n)$ and $\varphi(n) \mid ab - 1$, we can let $ab - 1 = t(p_i - 1)$ for some integer t. Since d is not relatively prime to n, there are two cases for each i. Either $p_i \mid d$, which makes the above equation obviously true, or $p_1 \nmid d$, in which case Fermat's little theorem tells us that

$$d^{ab} \equiv d^{ab-1} d \equiv d^{t(p_i-1)} d \equiv d \pmod{p_i}$$

As mentioned before, by the Chinese remainder theorem, since our congress is true modulo each p_i, it is true modulo n.

Of course, when d is not relatively prime to n, the set of powers of d modulo n is much smaller and thus the encryption will be slightly less secure. This doesn't happen to often though and there is still a large enough set of possibilities, so it's safe to ignore.

Last, we must solve the problem of a having to be relatively prime to $\varphi(n)$. To solve this, we can just always set $a = 2^{16} + 1$. Then, when generating n, if on the slim chance a is not relatively prime to $\varphi(n)$, we can just regenerate n until it is. Notice that it doesn't matter if everyone in the world chooses the same value of a for their public key, because the real security comes from the large random value of n and the fact that it is impossible to find b, the inverse of a, modulo a specific n.

4 The RSA Algorithm

To sum everything up, here is the condensed version of what we created above. This algorithm is actually known as RSA, one of the most famous and useful cyber security algorithms that we have invented.

Key Generation

1. Generate two large random prime numbers p and q. Note: to see how to generate large primes, look at the appendix.
2. Compute $n = pq$ and $\varphi(n) = (p-1)(q-1)$.
3. Pick a to be a somewhat small prime number such as $2^{16} + 1$. If a is not relatively prime to n, repeat steps 1 and 2.
4. Compute b, the inverse of a modulo $\varphi(n)$. This can be done using the Euclidean algorithm since we have both of these values.
5. Share your public key in the form (a, n) and keep your private key in the form (b, n).

Encryption

1. Convert your data into an integer, d. If your data is really large, you can split it up into sections and encode each section as an integer. For example, ABZ might become the integer 010226 if we encode each letter based on its position in the alphabet.
2. Use the public key, (a, n), to compute $d' \equiv d^a \pmod{n}$. d' is the encrypted data.

Decryption

1. Given encrypted data d', use the private key (b, n) to compute $d \equiv d'^b \pmod{n}$. d is the encoded form of the decrypted data.
2. decode d into it's original form. Using the example from before, $d = 010226$ might become ABZ.

Identity Verification

1. Person A and person B each generate a public-private key pair. They then share their public keys with each other.
2. Person A encrypts a bunch of random data with person B's public key. They send this encrypted data to person A and person B then decrypts it and sends the decrypted data back. Person A then verifies that person B was able to successfully decrypt all the data. Person A can then conclude that person B did in fact have the corresponding private key and has thus verified the identity of person B.
3. Person B now does the same for person A. In other words, repeat step 2, but with A and B swapped.

5 How Do We Verify Public Keys?

There's still one last problem with RSA. When you receive someone's public key, how do you know that that key truly corresponds to their identity? Well, in truth, there isn't an amazing answer. However, there are still many common techniques that are used in the real world:

1. The trust on first use (TOFU) model. When you connect to someone for the first time, you take it as a fact that the public key you get from them is correct and you store it for your own records. As long as a hacker didn't breach the first connection, the first key truly corresponds to whoever you are connecting to. Then, whenever you reconnect you can verify that the key is still the same as what you have stored. If it isn't, that means that the connection has been compromised.
2. Get keys in person. In order to avoid having to do all this digitally, you can just talk to whoever you want to connect to in person to be sure that they are giving you their key and that a digital hacker won't be able to intercept your conversation.
3. Use a certificate authority (CA). This is more practical for large numbers of connections. It would be impossible for each amazon customer to go talk to someone in the company to obtain a key. Instead, amazon has well known key that is verified by a trusted third party. Using a reliable third party verifier can be another way to ensure you have the correct public key for a specific connection.

6 Session Keys and the Diffie-Hellman Algorithm

It turns out that RSA encryption is actually rather slow. It works fine for a few probabilistic identity tests, but it will be noticeably slow for encrypted large files or messages. There is in fact a much faster encryption algorithm out there called AES. AES is a symmetric algorithm, meaning that it uses the same key for both encryption and decryption (of course, RSA is asymmetric). This type of key should be unique for every time you establish a connection, which is why it's called a session key. This is needed for added security so that if a hacker obtains one key, they can only decrypt one connection worth of messages.

Covering how AES works is not in the scope of this article and frankly, the algorithm just shifts data around in many convoluted ways to make it impossible to get the original data from the encrypted data. However, what is within the scope of this article is how two people can secretly share such a symmetric encryption key to use AES with.

It is tempting to turn to the obvious solution here: after verifying someones identity with RSA, you can simply decide on a random session key, encrypt it, and have the other person decrypt it. Although this works, it creates a problem: if a hacker can crack or find a flaw in our RSA algorithm, they can obtain our session keys as well. We want to separate out our layers of security whenever possible and not have them rely on each other since a chain is only as strong as its weakest link.

So, we have a new problem: If two people are communicating with each other without any prior communication, how do they generate a shared secret key such that if the third person hears everything they say, they will still not be able to decern the shared secret key.

The solution lies in the discrete logarithm problem: given g, r, and n, it is very difficult to find an a (assuming that such an a exists), such that $g^a \equiv r \pmod{n}$. Note that despite being similar, it is not the

same as the discrete root problem. Regardless, both are quite difficult to solve and sufficient for encryption. Here are the steps to follow:

1. Person A sends person B a prime number p and another positive integer g that is relatively prime to p.
2. Person A generates a random exponent a and person B generates a random exponent b.
3. Person A sends g^a to person B and person B sends g^b to person A.
4. Both person A and person B can now compute a shared secret key g^{ab} since $g^{ab} = (g^a)^b = (g^b)^a$.

Even if a third person listened to everything that person A and B said to each other, they would only obtain p, g, g^a, and g^b. In order to compute g^{ab}, this third person must either find a or b, but this is impossible because the discrete logarithm problem is infeasible to solve. The above algorithm is called the Diffie-Hellman key exchange algorithm and just like RSA, it is still commonly used today.

As always, there is one final catch to deal with. If we pick a bad value of g, there will be very few distinct powers of g modulo p. For example, if $p = 101$ and we pick $g = 10$, there will only be 4 distinct powers of g modulo p since the order of 10 modulo 101 is 4. To solve this problem, we have to pick p to be a safe prime. A safe prime is of the form $p = 2q + 1$ for another prime q. Thus, $\varphi(p) = 2q$. It follows that the order of any element is either 1, 2, q, or $2q$ (all the divisors of $\varphi(p)$). Then, as long as we don't pick any value of g with order 1 or 2, we can guarantee that g has at least q distinct powers modulo p. Eliminating the trivially bad values of g for safe primes p, we can make sure we pick a secure enough set of powers modulo p to share our secret key with.

It is important to note that safe primes are not easy to generate and quite rare. Thus, it is okay to use known safe primes which can be found in various pieces of documentation. This is because the modulus for this key exchange algorithm is not the secret. This is in contrast to RSA where the security lies behind knowing the prime factors of the modulus. In fact, always picking the same value of g is not a problem either since all the security lies in the randomness of a and b.

7 Conclusion

Cryptography is one of the coolest fields within computer science. It combines some of the most difficult concepts from number theory with simple questions about verifying information. At the same time, while designing all of these algorithms and thinking about all this math, we have to keep in mind the constantly changing landscape of computing and all the real world limitations that are present. Isn't it wild that this whole article, and so much more academic literature on this subject all came from the idea of sharing secrets and looking at ID cards? I, for one, think it is.

8 Appendix: Generating Large Primes and the Miller-Rabin Test

In order to implement RSA we must be able to generate large prime numbers. To prevent modern computational power, our primes must be anywhere from 1024 to 4096 bits in size. Common algorithms such as the sieve of Eratosthenes will not make even a dent in finding prime numbers of this size. Let's think of another way to test for primes.

Suppose n is a prime. Then, $a^{n-1} \equiv 1 \pmod{n}$ for any positive integer a less than n. However, the converse is not necessarily true. Moreover, the converse because even less likely to be true if we only check a few values of a. Just because this does end up being true for some a, it doesn't mean that n is prime.

However, we can hope that if we strengthen this test and apply it to a bunch of a, we will pretty accurately be able to determine whether n is a prime. Here is how we can strengthen the test: Let $n - 1 = 2^s * t$, where s and t are integers, and t is odd. Since the only square roots of 1 modulo a prime are ± 1, if n were prime, one of the following $s + 1$ conditions must be true

$$a^{2^i t} \equiv -1 \pmod{n} \quad \text{for some } 0 \leq i < s$$
$$a^t \equiv 1 \pmod{n}$$

This comes from taking the square root of the equation $a^{n-1} \equiv 1 \pmod{n}$ until the right hand side is no longer 1 or until the exponent of a is no longer even.

It turns out that when n is not prime, on average $\frac{1}{4}$ of the values of a less than n still satisfy these conditions. To be clear, whenever n is prime, this test will of course pass since one of those conditions will always be true. But, when n is not prime, this test may fail. That means that if we run this test for a bunch of randomly chosen values of a, say 100 values of a, the chance that we falsely identify a composite number as prime is $\frac{1}{4^{100}}$, which is essentially 0. There's a higher chance that a random bit flips and breaks all our algorithms anyway or that our computer systems simply shut down and break.

Isn't this crazy? The best way we know of generating large primes involves picking a random number, probabilistically checking if it's prime, and then incrementing it until it actually is prime. For all that we know about mathematics and about our world, it's rather beautiful that generating something as simple as primes takes an algorithm that's so clever and yet barbaric at the same time.